

ENSEA

Beyond Engineering

REAL-TIME OPERATING SYSTEMS

Session 4 – Software Timers

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Where we are

- The APP is an active object: it waits on its queue...
- ...but with a **10 ms timeout**: our home-made timers are still *polled*
  - The APP wakes up every 10 ms, even when nothing happens
  - A timer expires up to 10 ms late
- FreeRTOS provides **software timers**. Today, you discover them by yourselves

Format: flipped classroom

1. In pairs: read the documentation, answer the questions
2. Together: architecture, answers, limits and dangers
3. Read our timer service, rewritten on top of FreeRTOS timers

ENSEA
Beyond Engineering

Today

1 Questions

2 Architecture

3 Answers

4 Our Timer Service, Rewritten

ENSEA
Beyond Engineering

Documentation

- Principle: **Software timers**
  `https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/05-Software-timers/01-Software-timers`
- API: **Software Timer API Functions**
  `https://www.freertos.org/Documentation/02-Kernel/04-API-references/11-Software-timers/00-FreeRTOS-Software-Timer-API-Functions`

Goal

By the end, you should be able to draw how FreeRTOS timers work: *who* runs *what*, and *how* the information travels.

ENSEA
Beyond Engineering

Part 1 – Principle

1. What is a software timer? How does it differ from a hardware timer (TIM2, TIM3...)?
2. In which context does the callback run? What is it allowed to do?
3. When a task calls `xTimerStart`, does that task start the timer itself? How does the information reach the timer task?
4. What happens if that path is full? What is the `xTicksToWait` parameter of the API for?
5. Does the timer task run all the time? Does it use CPU when no timer expires?
6. What are `configUSE_TIMERS`, `configTIMER_TASK_PRIORITY`, `configTIMER_QUEUE_LENGTH`, `configTIMER_TASK_STACK_DEPTH` for? Which priority would you give the timer task?
7. One-shot or auto-reload: what is the difference?

ENSEA
Beyond Engineering

Part 2 – API

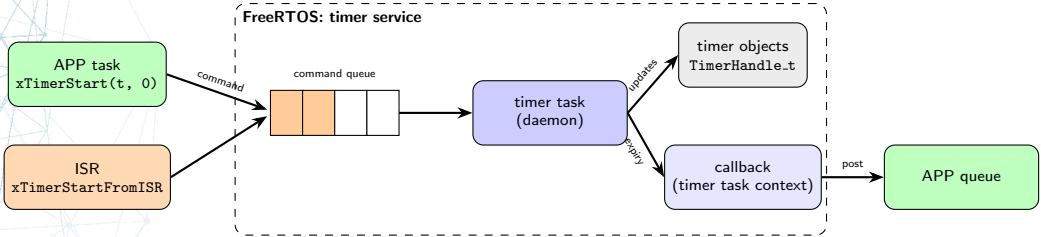
8. What does `xTimerCreate` do? What does it return? Where is the timer allocated? Does it start when created?
9. One callback shared by several timers: how do you know which one expired?
10. What is the difference between `xTimerStart`, `xTimerReset` and `xTimerChangePeriod`?
11. Can you start a timer from an interrupt? Before `vTaskStartScheduler()`?

Last question

Draw it: the task that starts the timer, the timer task, and the task that must be told the timer expired.

ENSEA
Beyond Engineering

How FreeRTOS timers work



- The API does not touch the timers: it sends **commands** into a queue
- One task – the **timer task** – owns the timers, processes the commands and runs the callbacks
- The timer task waits on its queue with a timeout = the next expiry

ENSEA
Beyond Engineering

Looks familiar?

Our APP

- Its own task
- A private queue
- A public interface: `app_post_event`
- Handlers run to completion

The FreeRTOS timer service

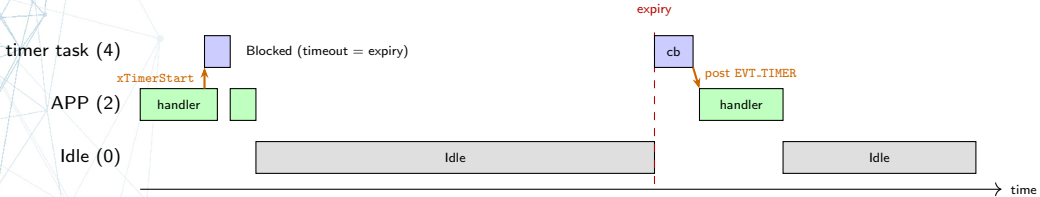
- Its own task (the timer task)
- A private queue (the command queue)
- A public interface: the `xTimer...` API
- Callbacks must be short and never block

The timer task *is* an active object

The pattern we built for our APP is the one FreeRTOS uses internally.

ENSEA
Beyond Engineering

In time



- The command preempts the APP (timer task priority $4 > 2$): it is processed immediately
- Between the two, nobody runs: the CPU is free, no polling
- At expiry, the callback runs in the timer task and **posts** an event: the work happens in the APP

ENSEA
Beyond Engineering

Answers (1/4)

1. **What is a software timer?**
   - A function (callback) executed later, managed by the kernel, with no dedicated hardware
   - Its resolution is the tick (1 ms here). A hardware timer is a peripheral: much finer, but limited in number
2. **In which context does the callback run?**
   - In the **timer task** (daemon, "timer service task"), not in the task that started the timer
   - It must be short and **never block**: no `vTaskDelay`, no waiting on a queue or semaphore
   - Otherwise every other timer is delayed
3. **Does the caller of `xTimerStart` start the timer?**
   - No: it sends a **command** into a queue, the **timer command queue**
   - The timer task processes it

ENSEA
Beyond Engineering

Answers (2/4)

4. **What if the command queue is full?**
   - The calling task blocks, up to `xTicksToWait`
   - With a timeout of 0, the call fails (`pdFAIL`): the command is **lost**
   - Trap: from a callback, i.e. from the timer task itself, waiting for room in its own queue = **deadlock**
5. **Does the timer task run all the time?**
   - No: it is Blocked on its queue, with a timeout equal to the next expiry
   - No timer running, no command: no CPU used

ENSEA
Beyond Engineering

Answers (3/4)

6 Configuration

- `configUSE_TIMERS`: enables the service (and creates the timer task)
- `configTIMER_TASK_PRIORITY`, `configTIMER_TASK_STACK_DEPTH`: the timer task
- `configTIMER_QUEUE_LENGTH`: the size of the command queue
- Low priority: callbacks are delayed by other tasks, the accuracy depends on it
- Callbacks are short and never block: the timer task can be **high** priority (session 1 rule)

7 One-shot or auto-reload?

- One-shot: expires once, then goes back to the dormant state
- Auto-reload: restarts automatically with the same period, after each expiry

ENSEA
Beyond Engineering

Answers (4/4)

8. `xTimerCreate`
   - Creates a timer object in the heap, returns a handle `TimerHandle_t` (`NULL` if no heap)
   - The timer is created **dormant**: it must be started
9. **Which timer expired?** The timer **ID**, set at creation, read with `pvTimerGetTimerID`
10. **Start, reset, change period**
    - `xTimerStart`: starts a dormant timer
    - `xTimerReset`: restarts the timer from now (starts it if dormant)
    - `xTimerChangePeriod`: changes the period **and** starts the timer
11. **From an ISR? Before the scheduler?**
    - ISR: the `FromISR` variants (never block)
    - Before the scheduler: yes, the command waits in the queue; the timeout is ignored

ENSEA
Beyond Engineering

Limits and dangers

- The callback runs in the **timer task**: never block, stay short – all timers share that task
- Access goes through a **queue**: a call with a timeout can **block** the caller if it is full
  - From a callback: risk of deadlock. Use a timeout of 0, and check the result
- Accuracy: one tick, plus the delay due to the priority of the timer task
- Stopping a timer does not cancel what its callback already did
  - An `EVT_TIMER` already posted in the APP queue can still arrive after `xTimerStop`
- Configuration: the priority of the timer task is fixed in `FreeRTOSConfig.h`, not chosen by `main`

ENSEA
Beyond Engineering

What changes, what does not

- `app/timers.{c,h}` renamed `app/app_timers.{c,h}`
  - Name clash with the `timers.h` of FreeRTOS
- The **interface does not change**
  - `app_timers_init`, `app_timer_start(id, deadline)`, `app_timer_stop`
  - `time_bomb` and `runner` are not modified: encapsulation pays off, again
- The implementation changes completely
  - One one-shot FreeRTOS timer per slot
  - No more `app_timers_process`: nothing to poll

ENSEA
Beyond Engineering

Creation and callback

```c
typedef struct {
    TimerHandle_t handles[APP_TIMER_COUNT];
} app_timers_t;

/* Runs in the timer task, not in the APP: short, never blocks */
static void app_timer_callback(TimerHandle_t handle) {
    int id = (int)(intptr_t)pvTimerGetTimerID(handle);
    app_post_event((app_event_t){.type = EVT_TIMER, .data = id});   /* timeout 0 */
}

void app_timers_init(app_timers_t *timers) {
    for (int i = 0; i < APP_TIMER_COUNT; i++) {
        /* one-shot, temporary period (set at each start), ID = slot number */
        timers->handles[i] = xTimerCreate("APP", 1, pdFALSE, (void *)(intptr_t)i,
                                          app_timer_callback);
        configASSERT(timers->handles[i] != NULL);
    }
}
```

- One callback for all the slots: the ID tells which one expired

ENSEA
Beyond Engineering

Start and stop

```
void app_timer_start(app_timers_t *timers, int id, uint32_t deadline_ms) {
    /* deadline -> delay; signed difference: safe at wraparound */
    int32_t delay_ms = (int32_t)(deadline_ms - bsp_get_tick_ms());
    TickType_t delay = (delay_ms > 0) ? pdMS_TO_TICKS(delay_ms) : 0;
    if (delay == 0) {
        delay = 1;                          /* a zero period is not allowed */
    }
    /* command to the timer task; timeout 0: the APP never blocks */
    BaseType_t ok = xTimerChangePeriod(timers->handles[id], delay, 0);   /* change + start */
    configASSERT(ok == pdPASS);
}

void app_timer_stop(app_timers_t *timers, int id) {
    BaseType_t ok = xTimerStop(timers->handles[id], 0);
    configASSERT(ok == pdPASS);
}
```

- Our API takes a **deadline** (HAL clock), FreeRTOS a **duration** (tick): converted when arming
- Timeout 0 (run-to-completion) and `configASSERT`: a lost command is never silent

ENSEA
Beyond Engineering

The APP task, finally

Session 2

```
for (;;) {
    if (xQueueReceive(app.events, &evt,
            pdMS_TO_TICKS(10)) == pdPASS) {
        app_handle_event(&evt);
    }
    app_timers_process(&app.timers,
                       bsp_get_tick_ms());
}
```

Session 4

```
for (;;) {
    xQueueReceive(app.events, &evt,
                  portMAX_DELAY);
    app_handle_event(&evt);
}
```

- The APP only wakes up when there is something to do: an input, or a timer
- The timers are as accurate as the tick, not as the polling period

ENSEA
Beyond Engineering

Configuration

```
#define configUSE_TIMERS                1
#define configTIMER_TASK_PRIORITY       ( 4 )   /* above the BSP (3) */
#define configTIMER_QUEUE_LENGTH        10
#define configTIMER_TASK_STACK_DEPTH    256     /* in words */
```

- Set in CubeMX (FreeRTOS, Config parameters), so that a regeneration keeps them
- Priority 4: the most urgent task – allowed, because its callbacks are short and never block
- An exception to our rule "`main` chooses the priorities": the timer task is created by the kernel

ENSEA
Beyond Engineering

Summary

- A software timer = a callback run later by the kernel; resolution = the tick
- The callbacks run in the **timer task**: short, never blocking
- The API sends **commands** through a queue: beware of the timeout if it is full
- The timer service is an **active object**: task + private queue + interface
- Our timer service keeps its interface; the APP no longer polls and blocks with `portMAX_DELAY`

ENSEA
Beyond Engineering